

Fast Inference of Neural Networks in FPGAs for LHC Triggering

Author: Karolina Podhorska

CID: 02230607

Supervisor: Dr. Alexander Tapper

Abstract: Field-programmable gate arrays (FPGAs) have become an ideal hardware platform for the extremely low-latency environments of particle detectors at the Large Hadron Collider (LHC) at CERN. Their ability to provide customizable compute units and highly parallel computation allows them to be tailored to specific applications. Trigger systems commonly adopt machine learning (ML) algorithms, such as neural networks (NNs), to classify particle jets in real time. These models can be implemented directly in FPGA hardware. The `hls4ml` tool enables this by converting trained ML models from common frameworks such as PyTorch or Keras into High-Level Synthesis (HLS) code based on C++, which can then be compiled into FPGA logic implementing the neural network [1]. Because these algorithms must operate under strict latency and resource constraints, various neural network compression techniques have been explored, including quantization-aware training and network pruning to remove redundant weights.

Keywords: Particle physics, FPGAs, `hls4ml`, Machine learning, Hardware acceleration

Contents

1	Introduction	1
2	Field-Programmable Gate Arrays (FPGAs)	2
2.1	FPGA Architecture	3
3	Accelerated Neural Networks	4
3.1	Key Terms	4
3.2	Design Techniques	5
3.3	hls4ml Library	6
4	Implementation	7
4.1	The Dataset	7
4.2	Benchmark Neural Network	8
4.3	Conversion to FPGA Firmware	10
4.4	Fixed-point Precision	10
4.5	Quantisation-aware Training	12
4.6	Pruning	13
5	Summary and Future Extensions	14
6	References	16

1 Introduction

The last few decades have seen a drastic increase in data production and collection, whether it be at big tech companies, traditional industries, governments, or scientific research centres. We have entered what many now refer to as the “Big Data” era, the term first introduced in the mid-1990s [2], indicative of the immense size of data produced with no efficient or realistic way of analysing it using standard statistical software [3]. A leading contributor to Big Data production is CERN, a centre of research home to the largest and most powerful particle accelerator in the world, the Large Hadron Collider (LHC) [4]. The LHC produces on average 30 petabytes of experimental data per year [5], and as of 2025, the CERN datacentre has collected and stored a total of one exabyte (10^{18} bytes) for analysis [6]. However, stored data only accounts for a small fraction of the total raw data generated in an experiment. Due to this sheer quantity of data and limitations in data storage capacity, a filter system is implemented in detectors to only select “interesting” events and store them. This filter is called the *trigger*, and its role is to discard away any raw events which it classified as background. Signal events, such as decays or rare particles, are stored for later analysis.

In the LHC, the CMS detector trigger system is composed of two levels: the level-1 (L1) trigger is implemented on customisable hardware, and is in charge of tentatively selecting candidate physics events [7], while the high-level trigger (HLT) uses software reconstruction and filtering algorithms to analyse full-granularity detector data [8]. Only data accepted by the HLT

are recorded for offline analysis. The L1 trigger operates at a fixed latency, and must make a decision in under $4\ \mu\text{s}$, using information from the calorimeter and muon detectors [9]. Candidate objects at the CMS include but are not limited to: muon ionization deposits, electron, photon, or tau lepton energy clusters, missing transverse energy (E_T), or jets.

Machine learning (ML) algorithms, such as neural networks (NNs) or boosted decision trees (BDTs), have a wide range of applications in event processing, jet identification, calibration, and offline analysis. For example, in CMS, a CSV algorithm based on deep machine learning was utilised for identification of heavy-flavour (charm and bottom) jets in proton-proton collisions at 13 TeV, which it succeeded at an efficiency of 68% [10]. In ATLAS, another algorithm based on BDTs was used to separate b -jets, c -jets and light-flavour jets in a $t\bar{t}$ sample [11]. These successes motivated the implementation of ML techniques within the L1 trigger [9]. These techniques rely on discriminatory variables measured at the detector to separate events into classes.

As a consequence of the extremely rigid and deterministic latency constraints, together with the high throughput demands of the LHC, Field-Programmable Gate Arrays (FPGAs) represent the most suitable platform for this application. FPGAs are highly customisable devices, which provide deeply pipelined designs that can be deployed in real-time systems. As a result, FPGAs have become a key component of the L1 trigger system, where multiple NN algorithms can run in parallel on each board.

The following sections outline the features and architecture of FPGAs, as well as NN implementation onto FPGA firmware. The `hls4ml` package, which translates ML algorithms to high-level synthesis code, is discussed, as well as common techniques for improving NN efficiency. Lastly, a case study is performed where a NN classifier is trained on an LHC jet-tagging dataset and compressed using different methods.

2 Field-Programmable Gate Arrays (FPGAs)

FPGAs are a type of integrated circuit that enable designers to directly program hardware behaviour at a granular level, making it possible to implement application-specific computation pipelines and integrated on-chip memory structures [12]. FPGAs have numerous features that distinguish them from general-purpose processors like CPUs or GPUs:

1. Firstly, they are completely customisable after deployment. Whereas other integrated circuits (MPUs, ASICs, MCUs) have a clearly specified function, FPGA hardware is programmable, which greatly enhances its capabilities and flexibility. FPGAs can be reconfigured with a completely new bitstream, which offers flexible adaptations during processing [12]. This flexibility increases the chip's lifespan and reduces the need for replacements, thus decreasing overall cost.
2. Another feature of FPGAs is their parallel processing capability. Parallelisation is the process of splitting calculations between multiple processing units (PUs), so operations are carried out simultaneously, and not sequentially.
3. Unlike CPUs and GPUs, which are constrained by fixed instruction sets and limited supported numerical precisions (e.g., int8, fp16), FPGAs enable the implementation of custom

compute architectures with arbitrary word lengths tailored to the application. [12].

2.1 FPGA Architecture

FPGAs consist of a matrix of configurable logic blocks (CLBs), or logic array blocks (LABs), linked by programmable interconnects. These programmable logic blocks contain various smaller components, including look-up tables (LUTs), flip-flops (FFs), and multiplexers. An LUT is a memory component which stores a list of outputs corresponding to different combinations of inputs. This means that any logic operation can be quickly accessed since the results are already stored in a truth table, and not recalculated. An n -input LUT uses n input signals to select one of 2^n possible outcomes. Most FPGAs use LUTs with 4-6 input bits [13]. An FF is a basic memory element which stores a single bit of information. FFs store this bit until they are triggered to change, which usually occurs at a specific transition of a clock signal. This means that FFs are synchronous devices which can all be coordinated by the same clock [14]. Lastly, a multiplexer is a circuit which takes multiple input signals (2^n) and selects one of them as the output.

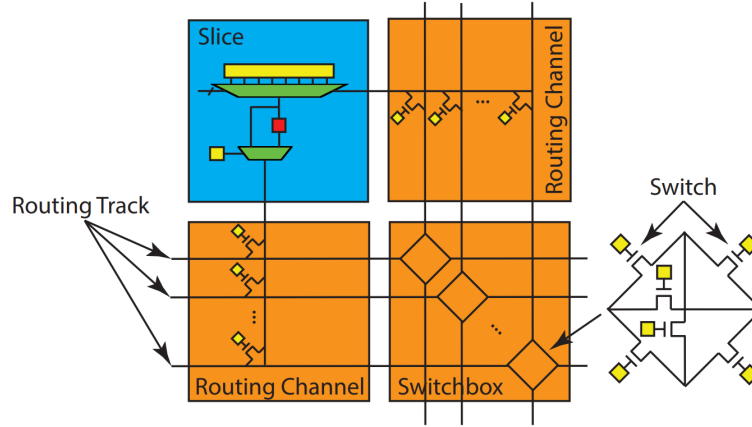


Figure 1: A slice containing one LUT and one FF, connected to other slices via routing channels and a switchbox. Figure taken from [13].

A small number of LUTs, FFs and multiplexers combined in a logic block is referred to as a *slice* [13]. Individual slices are joined by programmable interconnects, which are connected to a routing channel and a switchbox, as depicted in Figure 1. The routing channel implements a set of bits which can be configured to connect or disconnect slices from the programmable interconnect. The switchbox contains many switches which connect the routing tracks between individual routing channels. A 2-D diagram of an FPGA is shown in Figure 2, with slices, routing channels, and switchboxes. The purpose of the I/O blocks is communication with external devices.

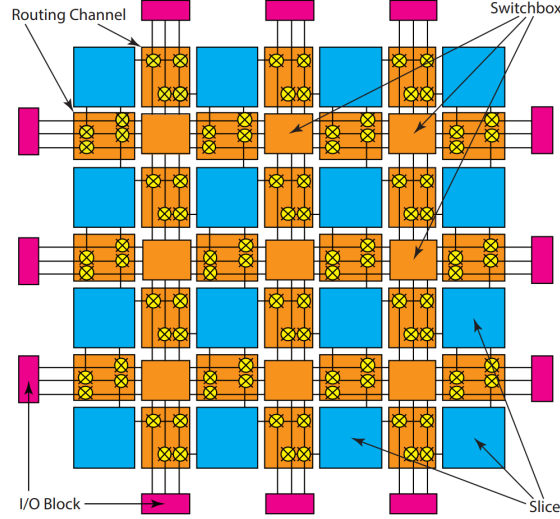


Figure 2: A diagram of a 2-D FPGA matrix of interconnected slices, routing channels, switchboxes and I/O Blocks, Figure taken from [13].

Many FPGAs incorporate Digital Signal Processor blocks (DSPs). These are specialised hard blocks that implement commonly performed operations, primarily Multiply-Accumulate (MAC) functions. These compute the product of two numbers and add that product to an accumulator in a single clock cycle. In a neural network, these functions are critical for applying kernels to input data, carrying out element-wise multiplications, and summing the results to create output feature maps [15]. Due to their dedicated hardened logic, DSPs are faster and more efficient than LUTs for these kinds of operations [16].

Another example of a hardened resource embedded in FPGAs is a Block Random Access Memory (BRAM). This is a RAM module which is fast, configurable, and deterministic, and is generally used to transfer data between the FPGA and store them on chip [13]. BRAMs are organised into fixed-size blocks, typically distributed in columns throughout the FPGA [12]. They can be configured to operate as single or dual port RAMs, depending on their design needs. One can also adjust their depth and width. BRAMs are used to store weights and biases and intermediate calculations in FPGA-implemented neural networks.

3 Accelerated Neural Networks

3.1 Key Terms

The performance of deep learning accelerators is typically quantified using two metrics: latency and throughput.

1. *Latency* is a key metric for real-time applications, and is defined as the time required for the accelerator to process a single input to obtain an output [12]. It is usually expressed in clock cycles or nanoseconds, and the aim with accelerated NNs is to achieve a lowest possible latency.
2. *Throughput* refers to the number of bits of data that is processed during one clock cycle,

expressed in bits per second [17]. The aim is to achieve high-throughput architectures. Sometimes the inverse quantity, the *initiation interval (II)* is used instead, which is a measurement of the number of clock cycles that pass before the algorithm can accept a new input.

3.2 Design Techniques

In order to maximise throughput and reduce latency, two main design techniques are implemented: pipelining and parallelisation [18]. Conventional computer processing platforms are based on serial architecture. This means that all arithmetic or logic operations are performed sequentially by the ALU (arithmetic logic unit). It is the rest of the CPU which supplies the ALU with data as fast as possible. The aim of the CPU is to perform a three-step process: fetch the instruction from memory, decode this instruction, and lastly execute it [19]. The speed of the CPU is limited by how fast data can be squeezed between memory and the ALU, also known as the von Neumann bottleneck [20].

An approach used to speed up this fetch-decode-execute loop is *pipelining*. This splits the three phases to occur simultaneously on separate processors. Essentially, while the first instruction is being executed, the second instruction is decoded, and the third instruction is being fetched [19]. This is visualised in Figure 3. Because multiple tasks are being executed at the same time, a pipelined design is much more efficient, offering a higher throughput than traditional sequential designs. Data pipelining also limits the need to store temporary results in memory before rereading them for subsequent processing. Instead, results are written to sets of local registers, where they may be accessed in parallel, improving data flow efficiency.

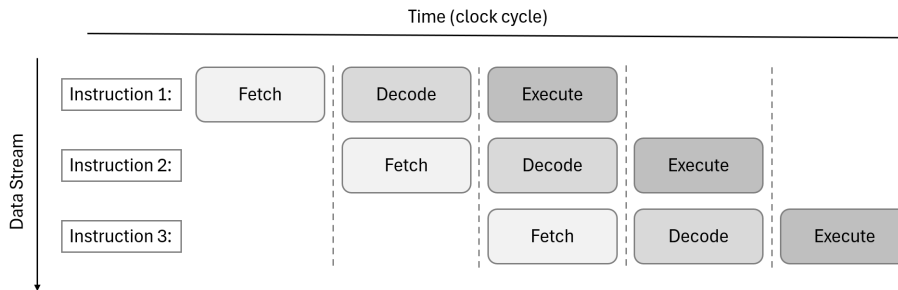


Figure 3: A pipelined architecture.

Another core technique implemented in FPGAs is *parallelisation*, where calculations are executed concurrently on several processing units [18]. For example, data (spatial) parallelisation involves partitioning the input data into smaller segments and using a separate processor to perform operations on each segment in parallel. On the other hand, task (temporal) parallelisation entails using a separate processor for each operation step, resembling a production line. This naturally leads to a pipelined structure [19]. Both types of parallelisation are depicted in Figure 4.

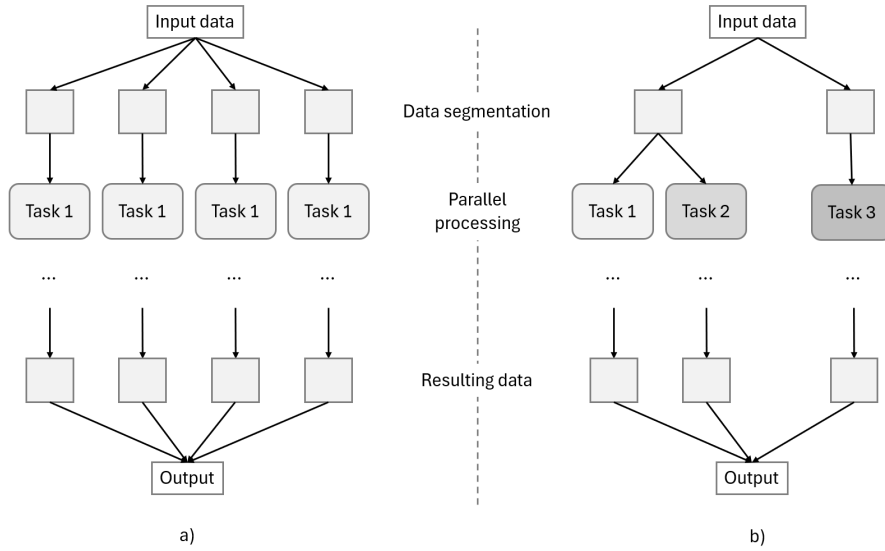


Figure 4: A diagram showing two types of parallelised architectures: a) data and b) task parallelisation.

However, if the algorithm is largely sequential, with each step dependent on the output of the preceding one, parallel execution provides little reduction in overall calculation time. For parallelisation to be effective, the algorithm must largely consist of operations that can be performed simultaneously. This limitation is known as Amdahl’s law (1967) [21], which states that the achievable speedup is fundamentally constrained by the fraction of the algorithm that remains sequential [19].

Although parallelisation and pipelining can accelerate portions of an algorithm, overall system performance is not determined solely by the achievable speedup. In hardware implementations, especially deeply pipelined architectures, performance is governed by a fundamental trade-off between latency and throughput [22]. In a pipelined design, the depth of the pipeline (i.e., the number of clock cycles required to process a single data element) determines the latency. Increasing this depth can improve clock frequency and throughput by shortening the critical path of each stage. However, this also increases the total number of cycles required to obtain an individual result. As a result, a core challenge in FPGA design is identifying an appropriate balance between pipeline depth, clock frequency, and resource utilisation, such that neither latency nor throughput is significantly compromised.

3.3 hls4ml Library

In order to simplify the implementation of trigger algorithms, a general library for converting pre-trained ML models into FPGA firmware has been developed, **hls4ml**. **hls4ml** is an open-source compiler package which translates machine learning model algorithms to high level synthesis (HLS) code which can be run on FPGAs [1]. After an ML model is developed using tools such as Pytorch or Keras, it must be tuned to run on FPGA resources as efficiently as possible, while achieving desired latency and throughput goals. This must be done before generating HLS code using **hls4ml**. There are three common tuning techniques: compression, quantisation, and parallelisation. All three are discussed below [23].

Compression is the process of reducing the overall size of the network. A common type of compression is pruning, where all connections with weights below a certain threshold are eliminated [24]. This approach reduces the number of multiplications that must be carried out, while retaining good performance.

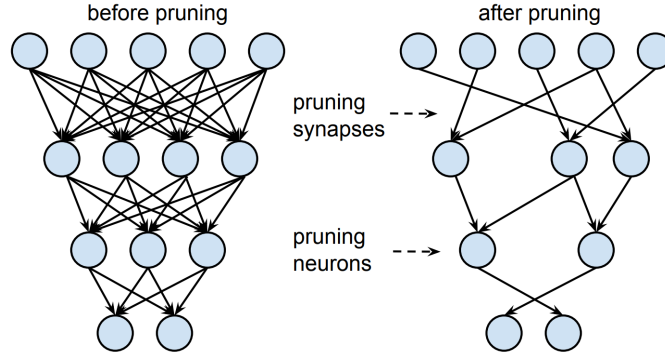


Figure 5: Pruning synapses and neurons, figure taken from [25]

Quantisation is the reduction in the number of bits required (precision) to represent each weight and bias [24]. As previously mentioned, FPGAs offer customisation of computation units. The `hls4ml` package uses fixed-point arithmetic as opposed to floating point [1]. A fixed-point representation consists of a fixed number of integer and fractional bits added together, typically expressed as $\langle \text{total bits, integer bits} \rangle$. This substitution improves energy efficiency as well as computational throughput without hurting performance [26].

Parallelisation is the process of performing calculations simultaneously to optimise throughput. The trade-off between latency, throughput, and computational resources is quantified in `hls4ml` using a “reuse factor” parameter, which describes the number of times each multiplier is used in a single layer. A reuse factor of one implies complete parallelisation, whereas a reuse factor of n uses a single multiplier n times sequentially [1]. Therefore, a low reuse factor signifies low latency and high throughput but uses most computation resources. A high reuse factor, on the other hand, implies longer latency and lower throughput, while saving more resources.

4 Implementation

4.1 The Dataset

For the following case study, OpenML’s `hls4ml_lhc_jets_hlf` jet-tagging dataset was used, available at [27]. This dataset includes 830,000 high- p_T simulated jet events, each with 16 corresponding features. Each event is labelled as one of five jet classes: light quark (q), gluon (g), W boson (W), Z boson (Z) and top quark (t). The 16 different observables can be broadly classed into “mass” observables (sensitive to the absolute jet energy) and “shape” observables (sensitive to the relative jet energies). The features are explained in more detail in [28]. By combining multiple features, one can achieve better overall jet tagging discrimination. The distribution of the five jet classes for each given observable feature are shown in Figure 6 below:

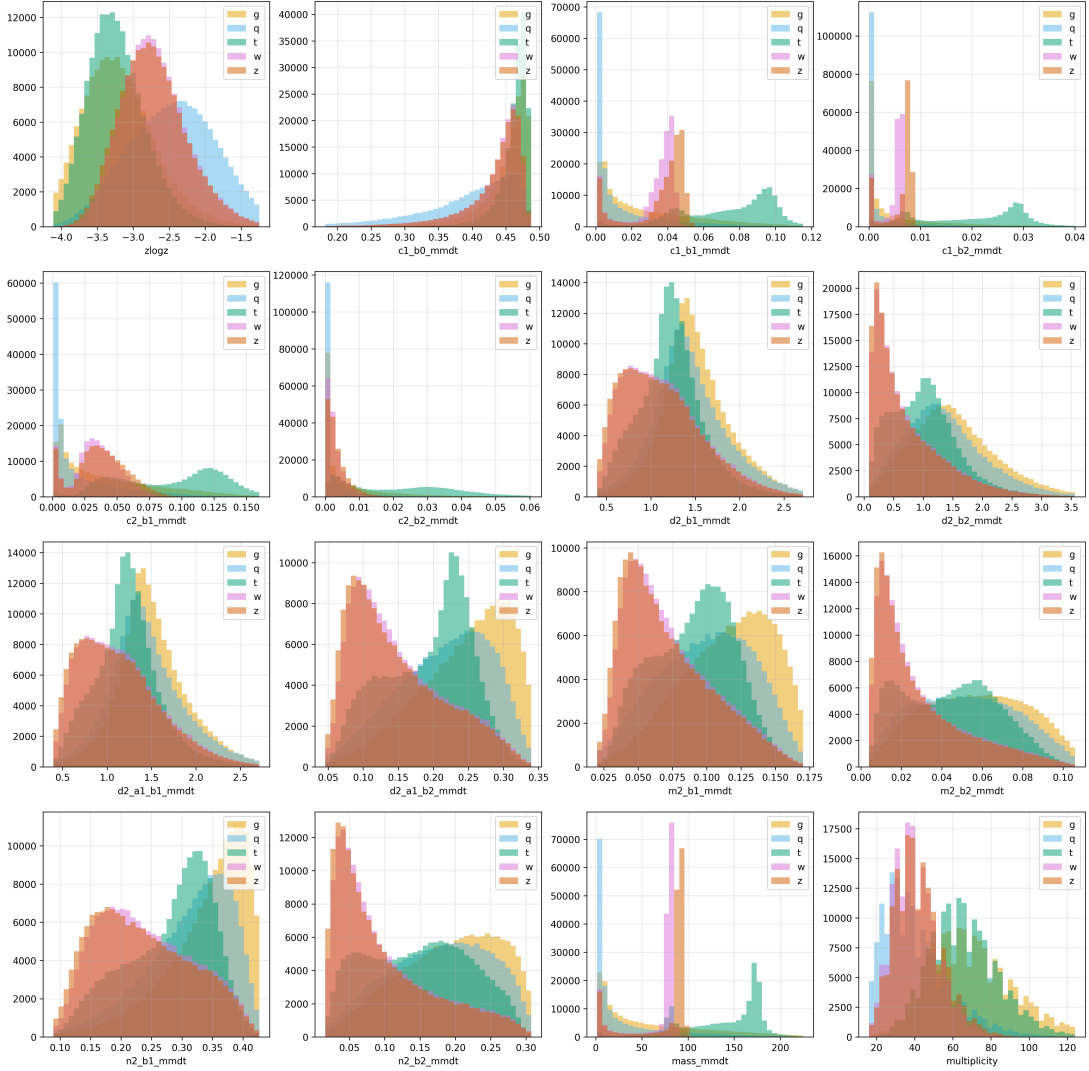


Figure 6: Histograms of the 16 recorded features, where each jet class is shown in a different colour.

4.2 Benchmark Neural Network

A `Keras` neural network shown in Figure 7 was trained to separate each event into the five distinct jet classes. This will serve as our benchmark floating-point performance metric, which will later be compared to our tuned networks. As part of data preprocessing, standardisation was performed by removing the mean and scaling to unit variance. This is a common requirement as estimators perform best when all data are normally distributed. This ensures large-magnitude features don't dominate the objective function, providing faster convergence and more stable training. The dataset was also split into training (98%) and testing (2%) subsets.

Next, a simple sequential model composed of a stack of fully connected layers was built with a `Tensorflow Keras` API. The network consists of three hidden layers, where each layer applies a weight and bias to the input, and produces an output. Before training, weights are initialised from a uniform distribution. An L1 regulariser is used to add a penalty to the kernel weights matrix. This promotes sparsity by removing smaller weights completely. A ReLU activation function is added after each fully connected layer to introduce nonlinearity to the network. At

the end, a softmax activation is applied to the outputs to convert the logits into probabilities for each class. Overall, this is a relatively small network, with 2821 tuneable parameters in total.

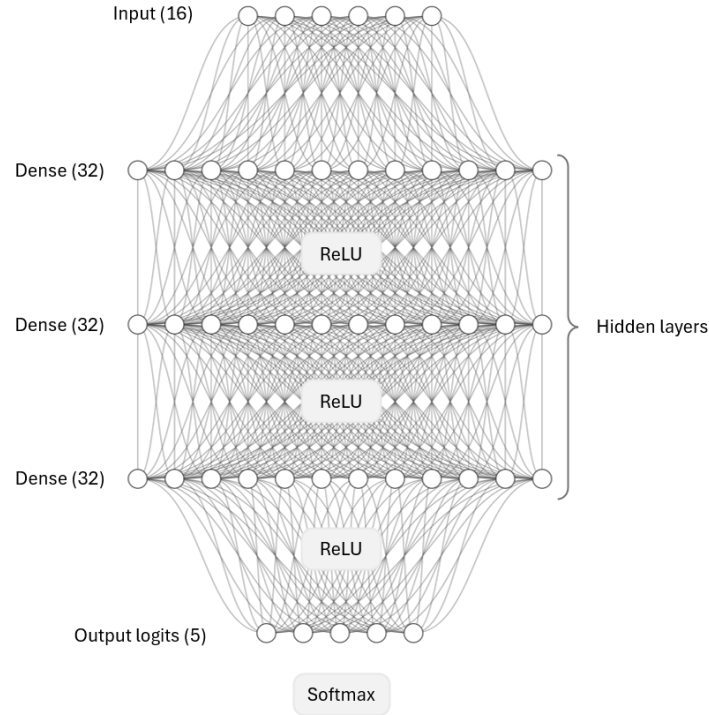


Figure 7: The structure of the baseline neural network.

The network was trained with a batch size of 128 over 20 epochs. A categorical cross-entropy loss function computes the loss after each epoch. The Adam optimiser with a learning rate of 0.0001 updates each weight using adaptive step sizes. After 20 epochs, it achieved a global accuracy of $\sim 76.1\%$ (subject to small fluctuations upon repeated training). This is defined as the fraction of correctly classified jets. The One-vs-Rest AUC was calculated for each jet class, as well as the average AUC. The results are shown in Table 1 below:

Table 1: Approximate AUC values of the floating-point `Keras` model.

Class	g	q	t	W	Z	Average
AUC	0.936	0.898	0.960	0.951	0.943	0.938

The jet class with the best separability was the top quark jet, likely due to its largest mass and shortest lifetime [29]. It's clear from Figure 6 that the top quark typically has a distinct distribution from the other classes for several features, so this result was expected. Below is a confusion matrix showing the relationship between the predicted and true class. It shows that light quarks and gluons sometimes get misidentified as each other, and the W and Z bosons sometimes get mistaken for light quarks.

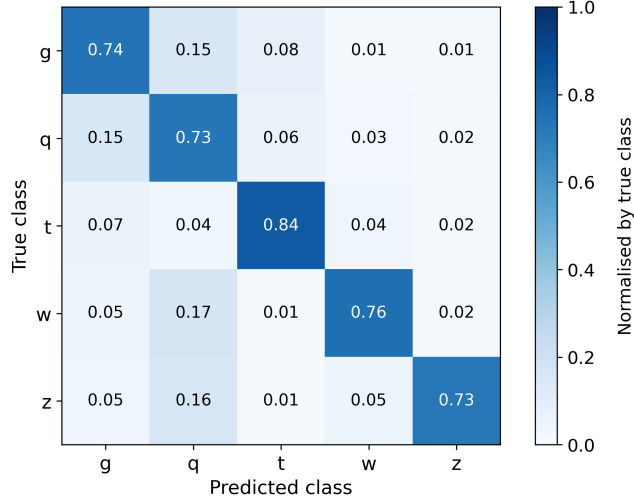


Figure 8: Confusion matrix constructed after testing the benchmark neural network.

4.3 Conversion to FPGA Firmware

In this section, the neural network was converted into hardware code which can be synthesised on an FPGA. The `hls4ml` package was used, with an Intel `OneAPI` backend. `hls4ml` generates C++ code implementing the neural network using fixed-point arithmetic. After compilation, a CPU-executable library is created that creates a bit-accurate emulation of the FPGA kernel. This step is essential for making comparisons and compressions before synthesising the design on an actual FPGA. The design was configured to run on an Intel Agilex7 FPGA board [30]. These are highest-performance FPGAs which offer high bandwidth and memory-intensive computations. An important thing to note is that the `OneAPI` backend only implements the **Resource** strategy for the layers, but no **Latency**. This means that it optimises for resource usage. [1]. Latency tests can only be conducted on a full simulation of the data flow within the FPGA. A reuse factor of 1 was set for this and all future builds.

4.4 Fixed-point Precision

As previously mentioned, `hls4ml` allows modification of the fixed-point precision at which internal data structures are stored. On one hand, using a lower precision could reduce FPGA resource usage. However, if the bit width is too narrow, computation accuracy may be sacrificed [1]. To demonstrate how bit width affects performance, the already-trained model created in Section 4.2 was converted to FPGA firmware with various precisions. The parameters are modified at inference time. For each fixed-point precision, the per-jet (one-vs-rest) AUC was calculated and compared to the original floating-point model. First, the number of fractional bits was kept constant at 8 while the integer bits were varied. Since the AUC ratio was constant at 1 at integer bit widths of 4 and greater, the fractional bit scan was performed for a fixed integer bit width of 4. Figure 9 shows the results for each jet class.

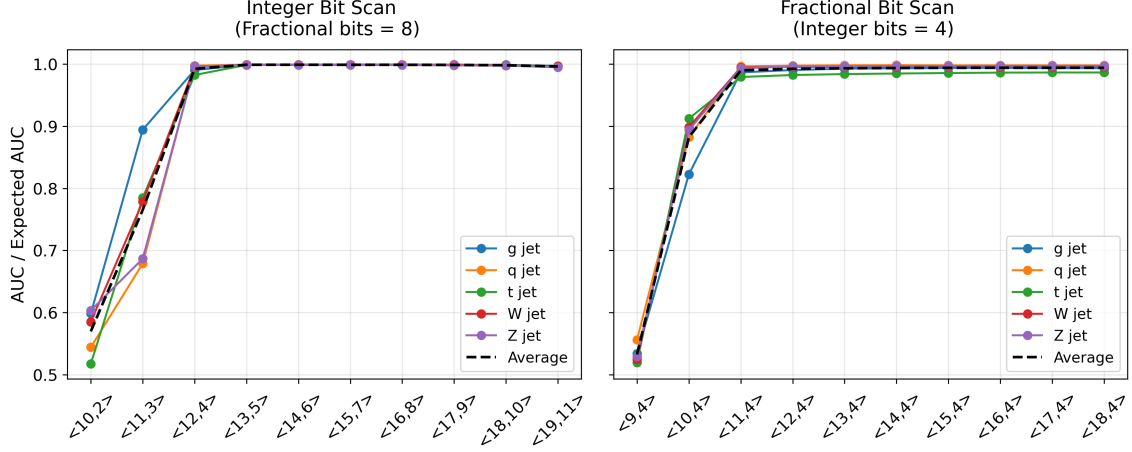


Figure 9: The (one-vs-rest) AUC ratio for the five jet classes as a function of bit width, expressed as $\langle \text{total bits}, \text{integer bits} \rangle$. (Left) the integer bit width is varied, while the fractional bit width stayed constant at 8. (Right) the fractional bit width was varied for a fixed integer bit width of 4.

The y-axis is equal to the ratio of the fixed-point model AUC and the floating-point AUC calculated in Section 4.2. A ratio of 1 signifies no loss in AUC when compared to the standard floating-point model. Both plots show a similar trend when the integer or fractional bit width was increased, eventually plateauing to a ratio of 1. However, as soon as the total bit width falls below 13, the AUC for each jet class was significantly reduced. This means that at lower bit widths the model did not have enough bits to properly store all data structures. This implies that optimal performance could be achieved with a $\langle 13, 5 \rangle$ or $\langle 13, 6 \rangle$ precision.

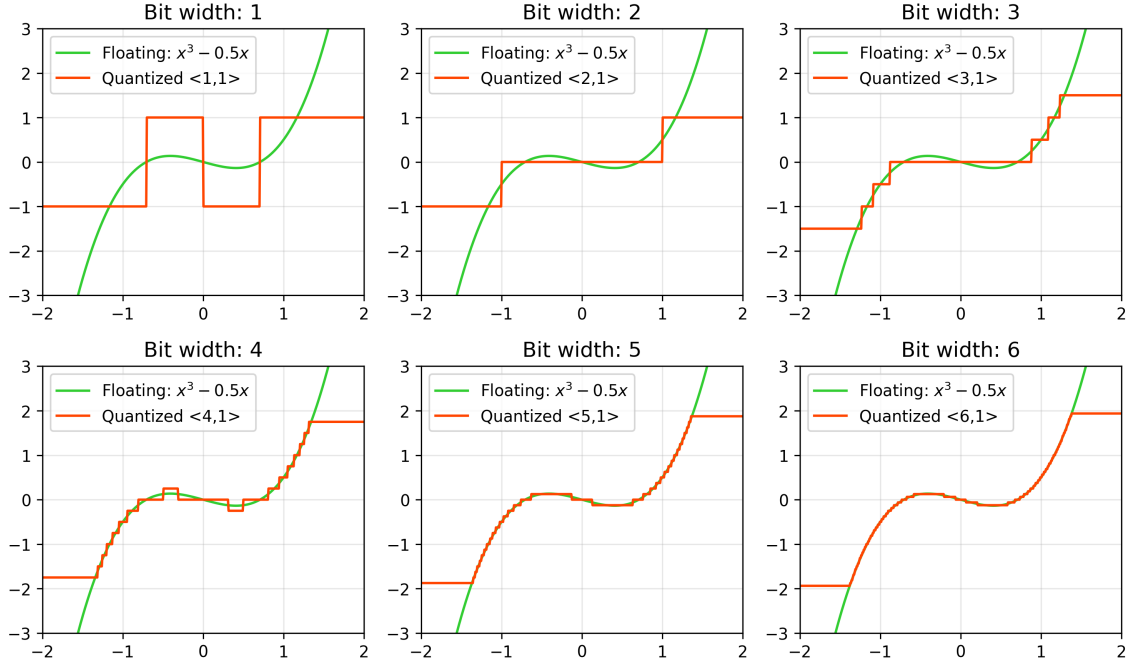


Figure 10: A demonstration of the effect of quantisation, comparing a cubic function at various bit widths.

4.5 Quantisation-aware Training

In Section 4.4, we have modified the fixed-point representation during inference. However, significantly lower bit widths could be achieved by quantising the model during training. This means that the network learns to classify jets within a certain bit width limit. To do this, the QKeras library was used, which is a quantisation extension to Keras [31]. All network layers (`Dense`) are replaced by their quantised counterparts (`QDense`), and a kernel and bias quantiser are implemented to set the total bit width while the integer bit width was fixed at 1. Quantising weights and biases implies that these variables can only take a specific array of possible values. For a bit width of $< n, 1 >$, the variable can only take one of 2^n values. This is demonstrated in practice in Figure 10, where a floating point cubic function is shown with varying degrees of quantisation. Already at a bit width of 6, the quantised curve matches the floating-point curve almost perfectly.

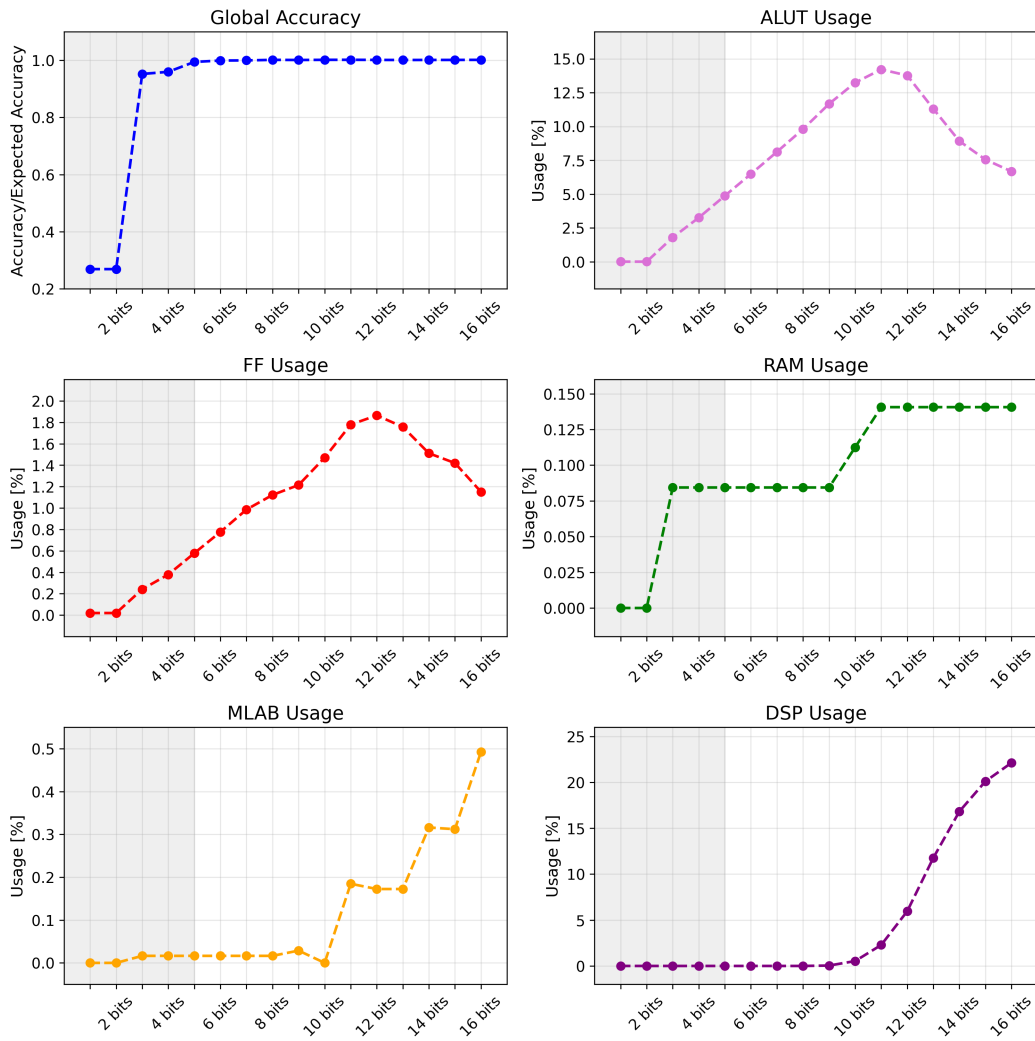


Figure 11: Accuracy and resource usage for quantisation-aware training, as a function of total bit widths. The shaded region signifies a fall in the accuracy ratio below 1.

A scan was conducted over total bit widths of 1-16, while keeping integer bit widths fixed at 1. The quantised network was trained over two epochs for each bit width. The global accuracy

ratio, ALUT, FF, RAM, MLAB and DSP percentage usage were all recorded for each bit width, the results shown in Figure 11. The shaded region corresponds to the values at which accuracy falls below a ratio of 1. These plots are representative of not only the model performance, but also the resource allocation at various precisions. Most importantly, the accuracy ratio stayed constant at 1 starting at bit widths as small as 4. This is a significant reduction in bit width, when compared to the standard `float32` model.

When it comes to resource usage, both ALUT and FF percentages increased steadily for the first 12 bit widths, but after that they started decreasing. At this turning point, DSP usage, which was stable at zero until now, started to gradually increase. This suggests that at larger bit widths, the operations got too complex for ALUTs and FFs, and hard blocks needed to be used instead. RAM and MLAB usage also increased as a function of bit width, but in drastic steps. This is because FPGA memory is stored in fixed blocks, not continuously. This means that when bit width increases past certain thresholds, the memory blocks must adapt their packing strategy to optimally allocate data, creating large jumps in usage. The key takeaway point is that quantisation substantially reduces resource usage, while maintaining a high global accuracy.

4.6 Pruning

In this section the network was compressed by iteratively removing connections based on their magnitude during training. This was done using the TensorFlow Model Optimization Toolkit, which includes a weight pruning API. The algorithm works by specifying the final target sparsity (the percentage of weights we want to set to zero), as well as a schedule to perform pruning. In our case, a constant sparsity schedule was implemented, which enforces a fixed sparsity level once pruning begins. Pruning was set to begin at 100 training steps, and was updated every 100 steps. The sparsity was set to 0, 0.25, 0.5, 0.75, 0.8, 0.85, 0.9, 0.95, 0.975, and 0.99, respectively. For each sparsity value, the model was trained, then exported to `hls4ml` for FPGA deployment. A necessary point to make here is that this is unstructured pruning, which only sets specific weights to zero, but doesn't remove them. This means that the tensor dimensions remain unchanged, but the network becomes more sparse.

The same variables were measured as in Section 4.5, and the results for each sparsity are shown in Figure 12. The global accuracy ratio stayed constant at 1 all the way up to 90% sparsity. This is an incredible result, because it implies that the network can achieve the same accuracy with 90% of its weights removed. The shaded region on each plot shows the sparsities for which the accuracy ratio dipped below 1. Both ALUT and FF usage steadily decreased as a function of sparsity, which implies that even though the number of multiplications stayed the same, the compiler must have eliminated trivial operations, such as multiplication by zero.

RAM and MLAB usage remained approximately zero across all sparsity levels. This indicates that the weight matrices were not implemented using block memory resources. Instead, the synthesis tool likely treated the trained weights as compile-time constants and embedded them directly into the generated logic as wires or registers. Similarly, DSP usage remained at zero across all sparsities. This indicates that the multiplications in the FPGA-synthesised design were implemented using LUT-based logic rather than dedicated DSP blocks. Since no DSP

resources were used in the baseline implementation, increasing sparsity could not reduce DSP utilisation.

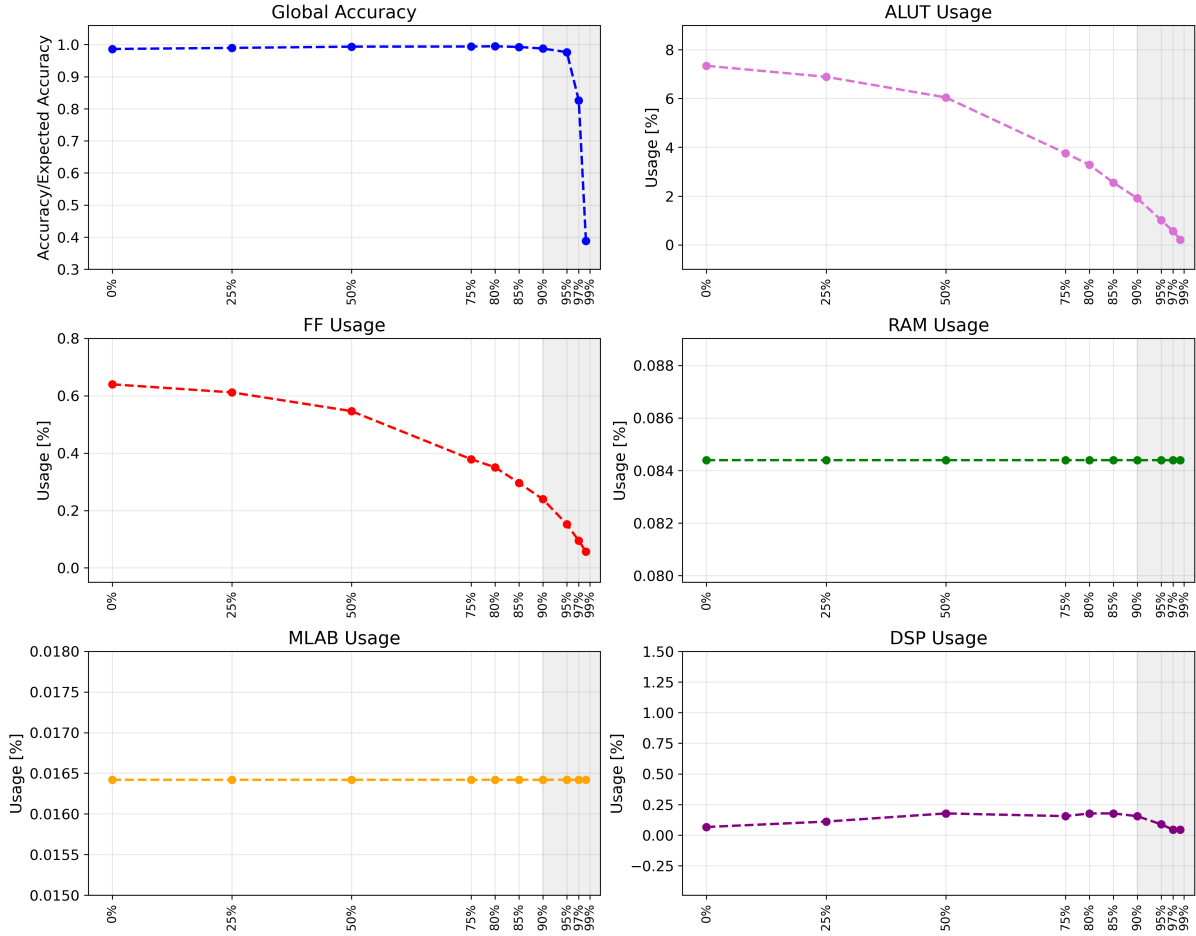


Figure 12: Accuracy and resource usage for different pruning sparsities. The shaded region signifies a fall in the accuracy ratio below 1.

5 Summary and Future Extensions

In this work we outlined the circumstances that led to the Big Data era, and how CERN’s substantial data collection warranted a need for a filtering system. The trigger was introduced, whose role is to filter the raw events at the detector so only interesting physics events are stored for later analysis. The core component of the trigger hardware is an array of FPGAs, which are highly customisable integrated circuits which enable parallel programming, as well as custom compute units. This makes them highly flexible for any application. The building blocks of FPGAs were described, and how neural networks can be implemented onto FPGA hardware. To do this, the `hls4ml` library is used, which converts `Keras` NNs into HLS code. An example of FPGA implementation of NNs which classify jets at the CMS detector was outlined. First, a benchmark `Keras` NN was built and its performance was measured. Next, it was converted to FPGA firmware using `hls4ml` with different fixed-point precisions. Two tuning techniques were explored: quantisation-aware training and pruning, and both were successful in significantly reducing resource usage while retaining global accuracy.

A possible extension to this work would explore combining quantisation-aware training with pruning, and analyse how this affects resource usage and performance. Similar work was done in ref. [32], where they applied deep compression through pruning, trained quantisation, and Huffman coding. They achieved a network size reduction by a factor of 49, without a loss of accuracy.

Lastly, a valuable direction for future work would be the deployment of the generated NN design on a physical FPGA device. The `hls4ml` framework only provides estimates of hardware resource usage through HLS reports, but these are not real measurements from an FPGA-implemented system. If instead we executed the design on an actual FPGA, more practical insight could be provided about the impact of various compression methods on the accuracy. In addition, it could reveal possible synthesis, logic optimisation or routing effects that cannot be captured by high-level resource estimates alone, since these occur later in the FPGA design flow.

6 References

- [1] Fast Machine Learning Lab. *hls4ml Documentation*. fastmachinelearning.org.
- [2] Steve Lohr. *The Origins of ‘Big Data’: An Etymological Detective Story*. *The New York Times*, 2013. archive.nytimes.com.
- [3] C.C.P. Snijders, U. Matzat, and U.D. Reips. *“Big Data” : big gaps of knowledge in the field of internet science*. *International Journal of Internet Science*, 2012. research.tue.nl.
- [4] Lyndon Evans and Philip Bryant. *LHC Machine*. *Journal of Instrumentation*, 2008. DOI:10.1088/1748-0221/3/08/S08001.
- [5] CERN Document Server. *Computing*. 2012. cds.cern.ch/record.
- [6] CERN. *CERN hits one exabyte of stored experimental data from the LHC*. 2025. home.web.cern.ch.
- [7] The CMS Collaboration. *The CMS trigger system*. *Journal of Instrumentation*, 2016. [arXiv:1609.02366](https://arxiv.org/abs/1609.02366).
- [8] Bauer, Gerry. *Recent experience and future evolution of the CMS High Level Trigger System*. *18th IEEE Real-Time Conference*, 2012. cds.cern.ch/record/1461735.
- [9] The CMS Collaboration. *The Phase-2 Upgrade of the CMS Level-1 Trigger*. *Technical Design Report CERN*, 2020. cds.cern.ch/record/2714892?ln=en.
- [10] CMS Collaboration. *Identification of heavy-flavour jets with the CMS detector in pp collisions at 13 TeV*. *Journal of Instrumentation*, 2018. [arXiv:1712.07158](https://arxiv.org/abs/1712.07158).
- [11] ATLAS Collaboration. *Measurements of b-jet tagging efficiency with the ATLAS detector using $t\bar{t}$ events at $\sqrt{s} = 13$ TeV*. *JHEP*, 2018. [arXiv:1805.01845](https://arxiv.org/abs/1805.01845).
- [12] Andrew Boutros, Aman Arora, Vaughn Betz. *Field-Programmable Gate Array Architecture for Deep Learning: Survey & Future Directions*. *Proceedings of the IEEE*, 2024. [arXiv:2404.10076](https://arxiv.org/abs/2404.10076).
- [13] Ryan Kastner, Janarбек Matai, Stephen Neuendorffer. *Parallel Programming for FPGAs*. *ArXiv e-prints*, 2018. [arXiv:1805.03648](https://arxiv.org/abs/1805.03648).
- [14] Ramsha Jawaaid. *Mastering Flip-Flop Circuits: Theory, Design, Types, and Real-World Applications*. 2025. www.wevolver.com.
- [15] Mihiraamsh, B. S. and Anita, J. P. *Analysis of Multiply-Accumulate (MAC) Unit Using Convolution Neural Networks (CNN)*, *Soft Computing: Theories and Applications*. Springer Nature Singapore, 2024. link.springer.com.
- [16] Jan Sommer, M. Akif Özkan, Oliver Keszocze, Jürgen Teich. *DSP-Packing: Squeezing Low-precision Arithmetic into FPGA DSP Blocks*. *32nd International Conference on Field-Programmable Logic and Applications (FPL)*, 2022. [arXiv:2203.11028](https://arxiv.org/abs/2203.11028).

- [17] Steve Kilts. *Advanced FPGA Design, Architecture, Implementation, and Optimization*. John Wiley and Sons, Ltd Publication, 2007. DOI:10.1002/9780470127896.
- [18] Jan-Frederik Schulte, Benjamin Ramhorst, Chang Sun, Jovan Mitrevski, Nicolò Ghielmetti, Enrico Lupi, Dimitrios Danopoulos, Vladimir Loncar and Javier Duarte. *hls4ml: A Flexible, Open-Source Platform for Deep Learning Acceleration on Reconfigurable Hardware*. ArXiv e-prints, 2025. arXiv:2512.01463 .
- [19] Donald G. Bailey. *Design for embedded image processing on FPGAs, 2nd edition*. John Wiley and Sons, Ltd Publication, 2024. DOI:10.1002/9780470127896.
- [20] Backus, John. *Can programming be liberated from the von Neumann style? A functional style and its algebra of programs*. Association for Computing Machinery, 1978. DOI:10.1145/359576.359579.
- [21] Amdahl, G. M. *Validity of the single processor approach to achieving large scale computing capabilities*. Spring Joint Computer Conference, Atlantic City, 1967.
- [22] Subhlok, Jaspar and Vondran, Gary. *Optimal latency-throughput tradeoffs for data parallel pipelines*. Association for Computing Machinery, 1996. DOI:10.1145/237502.237508.
- [23] J. Duarte, S. Han, P. Harris, S. Jindariani. *Fast inference of deep neural networks in FPGAs for particle physics*. Journal of Instrumentation, 2018. arXiv:1804.06913 .
- [24] Song Han, Huizi Mao, William J. Dally. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. Published as a conference paper at ICLR, 2016. arXiv:1510.00149 .
- [25] Song Han, Jeff Pool, John Tran, William J. Dally. *Learning both Weights and Connections for Efficient Neural Networks*. Published as a conference paper at NIPS, 2015. arXiv:1506.02626 .
- [26] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, Pritish Narayanan. *Deep Learning with Limited Numerical Precision*. ArXiv e-prints, 2015. arXiv:1502.02551 .
- [27] OpenML Contributors and LHC Jets HLF Curators. *hls4ml LHC JetsHLF (OpenML Dataset 42468)*. 2020. www.openml.org/d/42468.
- [28] Evan Coleman, Marat Freytsis, Andreas Hinzmann, Meenakshi Narain, Jesse Thaler, Nhan Tran, Caterina Vernieri. *The importance of calorimetry for highly-boosted jet substructure*. Journal of Instrumentation, 2018. arXiv:1709.08705 .
- [29] Michele Gallinaro. *Top quark physics: A tool for discoveries*. Journal of Physics: Conference Series, 2013. DOI 10.1088/1742-6596/447/1/012012.
- [30] altera. *Agilex 7 FPGAs and SoC FPGAs*. www.altera.com.
- [31] Claudionor Coelho. *QKeras*. 2019. github.com/google/qkeras .

- [32] Song Han, Huizi Mao, William J. Dally. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. Published as a conference paper at ICLR*, 2016. [arXiv:1510.00149](#) .